UNIVERSIDAD AUTÓNOMA DE CHIAPAS

ESCUELA DE TECNOLOGÍAS APLICADAS C-I

LICENCIATURA EN INGENIERÍA EN DESARROLLO Y TECNOLOGÍAS DE SOFTWARE

MATERIA:

INTELIGENCIA ARTIFICIAL

DOCENTE:

DR. GONZALEZ SCARPULLI DANIEL

PROYECTO INTEGRADOR 2a ENTREGA

PRESENTADO POR:

CLEMENTE CORZO JOSE RUBEN
GONZALEZ PEREZ YEINER YAIR
VARGAS OCAÑA LUIS FERNANDO
MALDONADO DE PAZ ELMAR ENRIQUE

7° M

TUXTLA GUTIÉRREZ, CHIAPAS

25 DE OCTUBRE DEL 2026

Introducción..... 3

Fase 1. Recuperación y actualización del planteamiento..... 4

Fase 2. Preparación de los datos, conocimiento o documentos..... 6

Fase 3. Selección definitiva de la técnica de Inteligencia Artificial..... 8

Fase 4. Diseño del componente inteligente..... 10

Fase 5. Implementación del prototipo..... 11

5.1 Código principal..... 11

5.2 Capturas de ejecución..... 12

5.3 Ejemplos de entradas..... 16

5.4 Resultados obtenidos..... 21

5.5. Descripción de las partes principales del programa..... 22

5.6 Explicación de dónde se encuentra el componente de Inteligencia Artificial.. 23

Fase 6. Pruebas del prototipo..... 24

Fase 7. Evaluación del desempeño..... 33

Fase 8. Análisis de errores..... 36

Fase 9. Relación con el agente inteligente propuesto..... 40

Fase 10. Riesgos, supervisión humana y uso responsable..... 42

Fase 11. Propuesta de mejora para la entrega final..... 43

Uso de Inteligencia Artificial generativa..... 46

Conclusiones..... 48

Referencias..... 49

Introducción

El acceso a la educación ha avanzado considerablemente, pero la calidad del aprendizaje enfrenta un problema crítico: la instrucción uniforme. Al presentar el mismo contenido, en el mismo orden y formato a todos los alumnos, se ignoran sus ritmos de aprendizaje y niveles de dominio, lo que fomenta el rezago y la deserción escolar.

Para atender esta necesidad social, este proyecto propone el desarrollo de un Sistema Inteligente de Recomendación Personalizada de Recursos y Rutas de Aprendizaje. Esta solución actúa como un motor adaptativo que no crea contenido nuevo, sino que recomienda de manera automatizada los recursos educativos óptimos según el estado cognitivo del estudiante. El propósito de esta segunda entrega parcial es presentar el diseño, implementación y evaluación inicial de un prototipo funcional de este sistema, demostrando, mediante experimentación técnica con datos simulados de telemetría y grafos de conocimiento, que la Inteligencia Artificial puede gestionar rutas dinámicas de aprendizaje y sugerir intervenciones pedagógicas precisas.

Fase 1. Recuperación y actualización del planteamiento

Para dar contexto a la implementación técnica desarrollada en esta segunda etapa, a continuación, se recuperan y actualizan los elementos fundamentales definidos en la entrega anterior:

1. **Nombre del proyecto:** Sistema Inteligente de Recomendación Personalizada de Recursos y Rutas de Aprendizaje.
2. **Problema específico:** Los estudiantes en entornos educativos convencionales carecen de un mecanismo que adapte dinámicamente el contenido, el formato y la secuencia de aprendizaje a su ritmo, nivel de dominio y estilo cognitivo individual, lo que genera deserción y brechas de conocimiento.
3. **Población o usuarios beneficiados:** Directamente, estudiantes de nivel medio superior, superior y autodidactas. Indirectamente, docentes e instituciones educativas.
4. **Entradas del sistema:** Perfil inicial del estudiante (nivel previo, formato preferido) y métricas de interacción (tema actual, tiempo de respuesta, porcentaje de errores, fallos consecutivos y abandonos).
5. **Resultado o acción esperada:** Determinar la acción pedagógica adecuada (avanzar, reforzar o retroceder) y generar dinámicamente la ruta de aprendizaje recomendando el recurso educativo óptimo para cada estudiante.
6. **Técnica de IA considerada inicialmente:** Grafos de conocimiento para modelar rutas y reglas lógicas de decisión (SI-ENTONCES) para el motor de inferencia.

Aspecto	Respuesta del equipo
¿El problema continúa siendo el mismo?	Sí, la necesidad de personalización dinámica del contenido frente a la instrucción uniforme sigue siendo el núcleo del proyecto.
¿Se modificó el alcance? ¿Por qué?	No se modificó el alcance general, pero para este prototipo funcional se delimitó la implementación a un entorno controlado usando un conjunto de datos sintéticos (telemetría simulada) y una base de conocimiento específica para el dominio de Desarrollo de Software (5 nodos), para calibrar el algoritmo sin comprometer datos reales sensibles.
¿La técnica de IA propuesta inicialmente continúa siendo adecuada?	Sí, el uso de Grafos de Conocimiento (con búsqueda A*) y reglas SI-ENTONCES demostró ser totalmente adecuado y necesario para gestionar la secuencialidad del aprendizaje y las decisiones pedagógicas (ej. retroceder ante fallos).
¿Qué cambios se realizaron respecto de la primera entrega?	Se enriqueció la propuesta inicial integrando un modelo de Aprendizaje Automático (Árbol de Decisión) para procesar la telemetría. Esta adición mantiene la naturaleza de "caja blanca" e interpretabilidad exigida en el ámbito educativo, operando en conjunto con el grafo y las reglas lógicas.

Fase 2. Preparación de los datos, conocimiento o documentos

Para el desarrollo de la primera versión funcional del Sistema Inteligente de Recomendación Personalizada de Recursos y Rutas de Aprendizaje, se utilizarán datos simulados que emulan los registros de telemetría de una plataforma LMS (como Moodle), así como una base de conocimiento estructurada para el dominio de Desarrollo de Software. Esto responde a la necesidad de calibrar el algoritmo antes de su despliegue con usuarios reales.

- **Fuente de los datos o conocimiento:** Dataset sintético generado específicamente para el prototipo (emulando telemetría de estudiantes) y un grafo de conocimiento creado manualmente que mapea los conceptos de desarrollo de software (prerrequisitos y temas).
- **Cantidad disponible:** 100 registros simulados de interacciones de estudiantes y un grafo estructurado con 5 nodos conceptuales (ej. Lógica, Programación Orientada a Objetos, Backend con Node.js, Bases de Datos).
- **Variables, atributos o características:** ID_Estudiante, Nivel_Previo (Principiante, Intermedio, Avanzado), Formato_Preferido (Visual, Texto, Práctico), Tema_Actual, Tiempo_Interaccion_Segundos, Tasa_Errores.
- **Variable objetivo, si existe:** Recurso_Recomendado_ID (Variable categórica que indica qué tipo de material y formato sugerir al estudiante para optimizar su aprendizaje).
- **Formato de los datos:** CSV para el dataset de telemetría y perfiles; y JSON/Diccionarios para la estructura del grafo de conocimiento.
- **Datos faltantes o inconsistentes:** Al ser un dataset sintético, se introdujeron deliberadamente valores nulos en la columna Tiempo_Interaccion_Segundos (un 5% de los datos) para simular escenarios reales donde la plataforma pierde conexión o el alumno deja la sesión inactiva.
- **Preparación o transformación realizada:** Se realizó limpieza de valores nulos (imputando la mediana para tiempos de interacción), codificación de variables categóricas (One-Hot Encoding para Formato_Preferido y

Nivel_Previo), y normalización de las métricas de telemetría continua para facilitar el procesamiento del algoritmo de clasificación.

- **Riesgos de privacidad o sesgo:** Existe el riesgo de sesgo algorítmico (efecto burbuja de filtro) si el modelo aprende a encasillar al estudiante exclusivamente en su formato preferido, reduciendo su exposición a nuevas modalidades cognitivas. Dado que se usan datos sintéticos en esta fase, no hay riesgo real de exposición de datos personales académicos sensibles.
- **Limitaciones de los datos:** Al ser información simulada, no logra capturar por completo la complejidad emocional, distracciones externas reales o frustración genuina del estudiante, factores que sí influyen en un entorno educativo vivo.

Evidencias de los datos y conocimiento:

	A	B	C	D	E	F	G
1	ID_Estudiante	Nivel_Previo	Formato_Preferi	Tema_Actual	Tiempo_Interacc	Tasa_Errores	Recurso_Recomendado_ID
2	1	Avanzado	Práctico	Backend con Node.js	138.0	17.88	Avanzado_Práctico_BACK
3	2	Principiante	Práctico	Lógica de Programación	67.0	36.65	Avanzado_Práctico_LÓGI
4	3	Avanzado	Práctico	Lógica de Programación	131.0	74.42	Refuerzo_Práctico_LÓGI
5	4	Avanzado	Visual	Programación Orientada a Objetos	10.0	72.09	Refuerzo_Visual_PROG
6	5	Principiante	Práctico	Backend con Node.js	248.0	30.81	Avanzado_Práctico_BACK
7	6	Principiante	Práctico	Backend con Node.js	299.0	54.25	Avanzado_Práctico_BACK
8	7	Avanzado	Visual	Programación Orientada a Objetos	105.0	50.88	Avanzado_Visual_PROG
9	8	Intermedio	Práctico	Bases de Datos PostgreSQL	135.0	63.63	Refuerzo_Práctico_BASE
10	9	Avanzado	Visual	Lógica de Programación	127.0	25.05	Avanzado_Visual_LÓGI
11	10	Avanzado	Texto	Patrón MVC en PHP	57.0	58.99	Avanzado_Texto_PATR
12	11	Avanzado	Práctico	Lógica de Programación	98.0	97.89	Refuerzo_Práctico_LÓGI
13	12	Avanzado	Texto	Lógica de Programación	246.0	48.67	Avanzado_Texto_LÓGI
14	13	Principiante	Visual	Bases de Datos PostgreSQL	281.0	90.61	Refuerzo_Visual_BASE
15	14	Avanzado	Práctico	Lógica de Programación	198.0	43.44	Avanzado_Práctico_LÓGI
16	15	Intermedio	Visual	Programación Orientada a Objetos		35.01	Avanzado_Visual_PROG
17	16	Principiante	Texto	Programación Orientada a Objetos	78.0	64.51	Refuerzo_Texto_PROG
18	17	Intermedio	Visual	Backend con Node.js	287.0	66.89	Refuerzo_Visual_BACK
19	18	Intermedio	Práctico	Patrón MVC en PHP	256.0	86.42	Refuerzo_Práctico_PATR
20	19	Intermedio	Práctico	Lógica de Programación	85.0	23.02	Avanzado_Práctico_LÓGI
21	20	Intermedio	Texto	Lógica de Programación	163.0	49.92	Avanzado_Texto_LÓGI
22	21	Principiante	Visual	Bases de Datos PostgreSQL	153.0	57.2	Avanzado_Visual_BASE
23	22	Principiante	Práctico	Programación Orientada a Objetos	95.0	76.86	Refuerzo_Práctico_PROG
24	23	Intermedio	Texto	Patrón MVC en PHP	194.0	4.36	Avanzado_Texto_PATR
25	24	Intermedio	Práctico	Backend con Node.js	294.0	99.46	Refuerzo_Práctico_BACK
26	25	Principiante	Práctico	Programación Orientada a Objetos	229.0	46.99	Avanzado_Práctico_PROG
27	26	Principiante	Visual	Backend con Node.js		27.96	Avanzado_Visual_BACK
28	27	Principiante	Práctico	Bases de Datos PostgreSQL	56.0	88.35	Refuerzo_Práctico_BASE
29	28	Avanzado	Visual	Bases de Datos PostgreSQL	103.0	74.77	Refuerzo_Visual_BASE



```
    },
    "Programacion_Orientada_a_Objetos": {
        "prerrequisitos": [
            "Logica_de_Programacion"
        ],
        "recursos": [
            "Recurso_Video_POO",
            "Recurso_Lectura_POO",
            "Recurso_Practica_POO"
        ]
    },
    "Bases_de_Datos_PostgreSQL": {
        "prerrequisitos": [
            "Logica_de_Programacion"
        ],
        "recursos": [
            "Recurso_Video_DB",
            "Recurso_Practica_DB"
        ]
    },
    "Backend_con_Nodejs": {
        "prerrequisitos": [
            "Programacion_Orientada_a_Objetos",
            "Bases_de_Datos_PostgreSQL"
        ],
        "recursos": [
            "Recurso_Video_Node",
            "Recurso_Practica_Node"
        ]
```

Fase 3. Selección definitiva de la técnica de Inteligencia Artificial

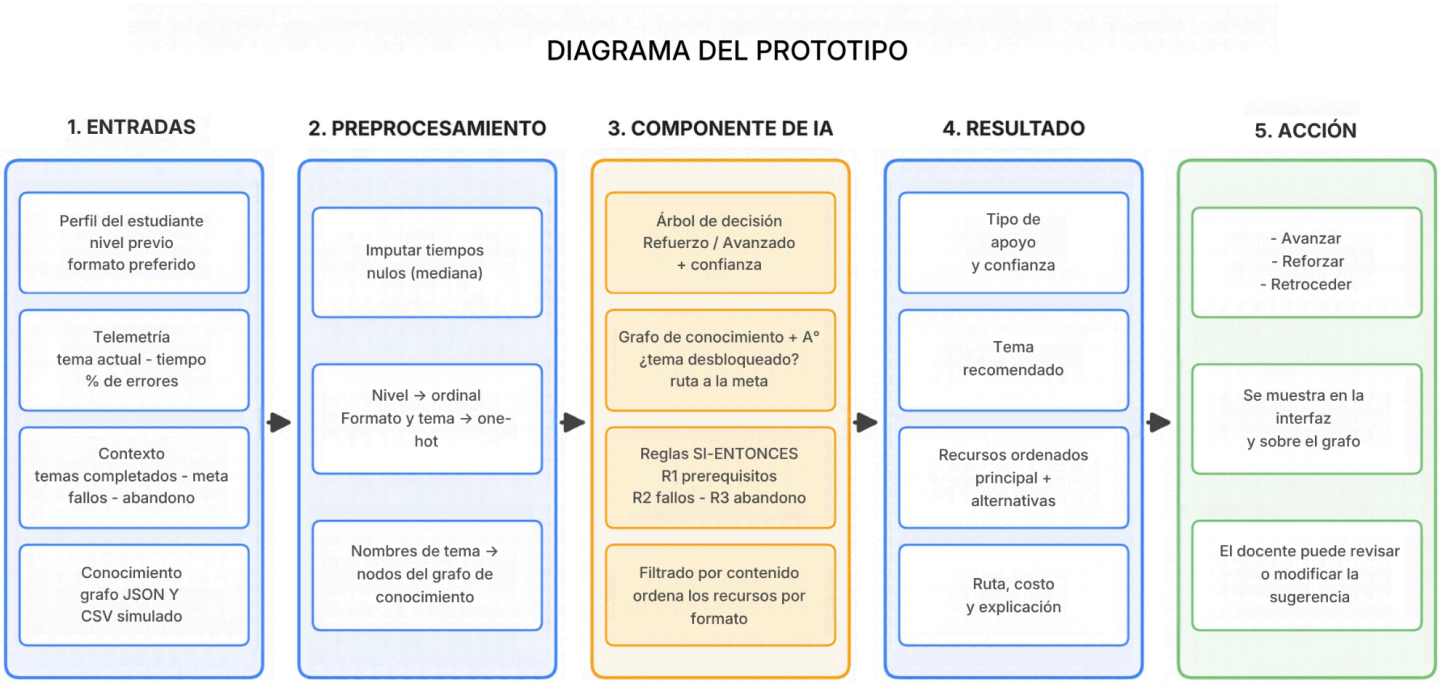
En concordancia con el modelo conceptual y la representación del conocimiento planteados en la Fase 4 de la primera entrega, se ha decidido implementar un enfoque híbrido para este primer prototipo funcional.

- **¿Qué técnica de IA utilizarán?** Se utilizará un **Árbol de Decisión (Clasificación)** combinado con una estructura de **Grafos de Conocimiento** (implementado mediante estructuras de red/búsqueda).
- **¿Qué tarea realizará?** El Grafo de Conocimiento gestionará las reglas de secuencia (determinar qué temas o nodos están desbloqueados según los prerrequisitos). El Árbol de Decisión procesará la telemetría y el perfil del alumno para clasificar y predecir cuál es el formato o recurso educativo específico más adecuado para el tema en curso.
- **¿Por qué es apropiada para el problema?** El aprendizaje no es lineal, por lo que el grafo permite mapear rutas dinámicas. Por su parte, el Árbol de Decisión es un modelo de caja blanca (altamente interpretable). Esto es fundamental en educación: permite a los docentes auditar las reglas lógicas

(SI-ENTONCES) para comprender *por qué* el sistema recomendó un video de repaso o sugirió avanzar al siguiente tema.

- **¿Qué entradas necesita?** El perfil inicial del estudiante (formato preferido, nivel de dominio) y los datos de telemetría de interacción simulada (tasas de aciertos/errores y tiempo de respuesta).
- **¿Qué resultado produce?** La habilitación del siguiente nodo del grafo (tema) y la recomendación priorizada del ID de un recurso pedagógico (ej. Video_05_MVC_PHP o Ejercicio_Practico_Node).
- **¿Qué otras técnicas podrían utilizarse?** Sistemas de recomendación puramente colaborativos (como KNN), Clasificadores de Naive Bayes, o arquitecturas complejas de Redes Neuronales Profundas (Deep Learning).
- **¿Por qué se eligió esta opción?** Se descartaron las Redes Neuronales debido a su naturaleza de "caja negra"; en el ámbito educativo planteado, es un requisito estricto que las decisiones mantengan supervisión y explicabilidad humana. El Árbol de Decisión emula directamente el motor de reglas de producción propuesto originalmente y resulta ser la opción más viable y transparente para integrar con el mapeo del grafo de conocimiento en esta etapa de prototipado.

Fase 4. Diseño del componente inteligente



El sistema recibe cuatro tipos de entradas: el perfil del estudiante (nivel previo y formato preferido), la telemetría de su interacción (tema que estudia, tiempo de interacción y porcentaje de errores), el contexto de su avance (temas completados, meta de aprendizaje, fallos consecutivos y si abandonó la actividad) y dos fuentes de conocimiento: un conjunto de 100 registros de telemetría simulada y un grafo de conocimiento con cinco temas y sus prerrequisitos, en el preprocesamiento se imputan con la mediana los tiempos faltantes, el nivel se codifica de forma ordinal, el formato y el tema se codifican con one-hot, y los nombres de los temas se traducen a los nodos del grafo.

La inteligencia artificial interviene en la tercera etapa, con cuatro piezas que trabajan en conjunto. Un árbol de decisión clasifica la situación del estudiante como Refuerzo o Avanzado. Un grafo de conocimiento con búsqueda *A** determina si el tema está desbloqueado y calcula la ruta de menor costo hacia la meta. Las reglas SI-ENTONCES resuelven casos particulares: prerrequisitos pendientes, fallos repetidos y abandono con poco tiempo de interacción. El filtrado

por contenido ordena los recursos según el formato preferido. El resultado es un tipo de apoyo, un tema recomendado, una lista ordenada de recursos y una ruta con su explicación. Se presenta en la interfaz como una acción sugerida (avanzar, reforzar o retroceder) y queda sujeto a la revisión.

Fase 5. Implementación del prototipo

5.1 Código principal

El código fuente completo del prototipo, así como los archivos de datos y el cuaderno de pruebas están en el enlace de Google Drive

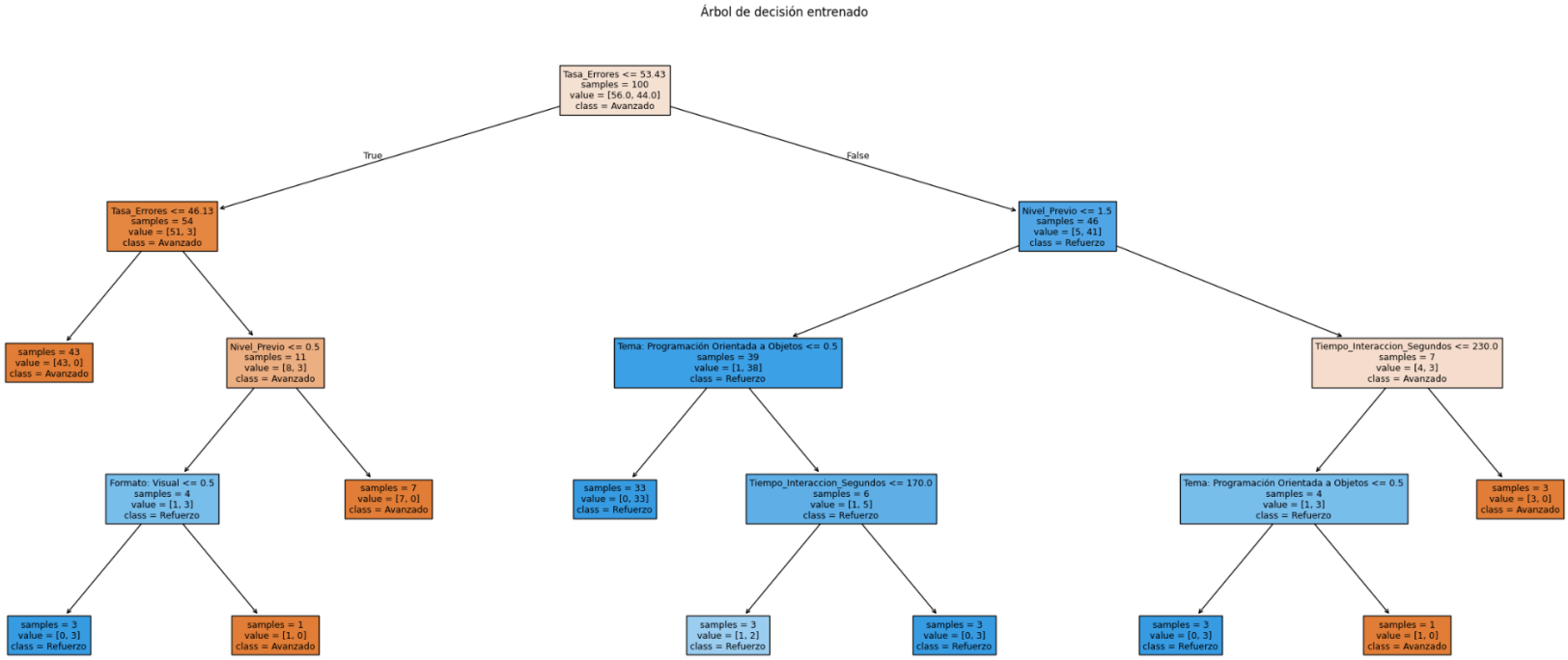
Archivo	Propósito
recomendador.py	Contiene toda la lógica del sistema: preprocesamiento, el Árbol de Decisión, el Grafo con el algoritmo A*, las reglas SI-ENTONCES, el filtrado por contenido y la interfaz gráfica (Gradio).
prototipo_recomendador.ipynb	Notebook principal (para Google Colab) que ejecuta el sistema paso a paso, muestra las métricas y genera las gráficas.
app.py	Script que lanza únicamente la interfaz gráfica web (python app.py).
telemetria_estudiantes_simulada.csv	Dataset con registros simulados.
grafo_conocimiento.json	Archivo JSON con los temas, sus prerrequisitos y recursos

requirements.txt	Archivo con las librerías necesarias para ejecutar el código (pandas, scikit-learn, networkx, gradio, etc.).
generar_datos.py	Genera el dataset csv simulado de telemetría con semilla fija



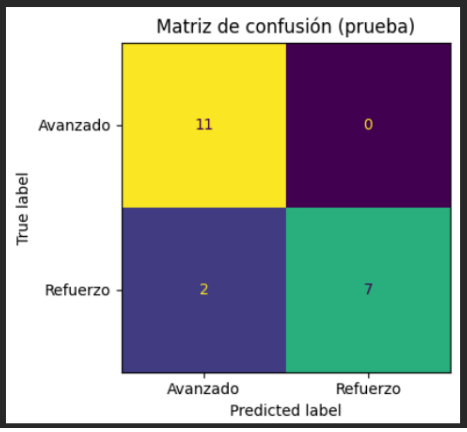
5.2 Capturas de ejecución

Entrenamiento del modelo de clasificación (Árbol de Decisión).



```python
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay(confusion_matrix(y_te, pred, labels=arbol.classes_), display_labels=arbol.classes_).plot(ax=ax, colorbar=False)
ax.set_title("Matriz de confusión (prueba)")
plt.tight_layout(); plt.show()

nombres_cortos = [n.split("__", 1)[1].replace("Tema_Actual_", "Tema: ").replace("Formato_Preferido_", "Formato: ") for n in nombres]
fig, ax = plt.subplots(figsize=(24, 10))
plot_tree(arbol, feature_names=nombres_cortos, class_names=list(arbol.classes_), filled=True, impurity=False, ax=ax, fontsize=9)
ax.set_title("Árbol de decisión entrenado")
plt.tight_layout(); plt.show()
```



Cálculo de rutas de aprendizaje mediante el algoritmo A* en el grafo de conocimiento

4. Componente de IA #2: grafo de conocimiento y búsqueda A*

El JSON en `ruta_a_estrella()` busca la secuencia de menor costo para llegar a una meta partiendo de lo que el estudiante ya completó. El costo de cada tema es `dificultad × factor del nivel` (supuestos definidos por el equipo en `recomendador.py`).

markdown

```python
print("Prerrequisitos del grafo:")
for p, t in rec.aristas:
    print(f"  {R.NODO_A_TEMA[p]}  →  {R.NODO_A_TEMA[t]}")

for nivel in R.NIVELES:
    ruta, costo, expl = R.ruta_a_estrella(rec.grafo, set(), "Backend_con_Nodejs", nivel)
    print(f"\n{nivel:12s} | costo = {costo:5.1f} | estados explorados = {expl}")
    print("   ", " → ".join(R.NODO_A_TEMA[n] for n in ruta))
```

Python

```
Prerrequisitos del grafo:
  Lógica de Programación  →  Programación Orientada a Objetos
  Lógica de Programación  →  Bases de Datos PostgreSQL
  Programación Orientada a Objetos  →  Backend con Node.js
  Bases de Datos PostgreSQL  →  Backend con Node.js
  Programación Orientada a Objetos  →  Patrón MVC en PHP

Principiante | costo =  18.0 | estados explorados = 6
    Lógica de Programación → Programación Orientada a Objetos → Bases de Datos PostgreSQL → Backend con Node.js

Intermedio   | costo =  12.0 | estados explorados = 6
    Lógica de Programación → Programación Orientada a Objetos → Bases de Datos PostgreSQL → Backend con Node.js

Avanzado     | costo =   8.4 | estados explorados = 6
    Lógica de Programación → Programación Orientada a Objetos → Bases de Datos PostgreSQL → Backend con Node.js
```

Interfaz gráfica interactiva desarrollada con Gradio mostrando la recomendación y el grafo.

Nivel previo

(•) Principiante ( ) Intermedio ( ) Avanzado

Formato preferido

(•) Visual ( ) Texto ( ) Práctico

Tema que está estudiando

Lógica de Programación

Tiempo de interacción (s) 120

5 300

Tasa de errores (%) 50

0 100

Temas ya completados

[ ] Lógica de Programación [ ] Programación Orientada a Objetos

[ ] Bases de Datos PostgreSQL [ ] Backend con Node.js [ ] Patrón MVC en PHP

Meta de aprendizaje

Backend con Node.js

Fallos consecutivos en el tema 0

0 6

[ ] Abandonó la actividad a la mitad

Recomendar

Acción sugerida: **Avanzar**

Tema recomendado: Lógica de Programación
Clasificación del árbol de decisión: Avanzado · confianza 100%

Recursos recomendados

1. `Recurso_Video_Logica` — Visual (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Practica_Logica` — Práctico (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_Logica` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1	Lógica de Programación	`Recurso_Video_Logica`	Visual
2	Programación Orientada a Objetos	`Recurso_Video_POO`	Visual
3	Bases de Datos PostgreSQL	`Recurso_Video_DB`	Visual
4	Backend con Node.js	`Recurso_Video_Node`	Visual

Costo de la ruta: 18.0 · estados explorados: 6

Reglas activadas

- Ninguna regla de excepción se activó; se usó solo la clasificación del árbol.

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Avanzado» (confianza 100%) con tasa de errores 50 %.

> Sugerencia adaptativa: el docente puede revisarla o modificarla. No constituye una calificación ni diagnóstico definitivo.

Grafo de conocimiento: flechas = prerrequisito · PASO n = orden de estudio



Gradio con Regla

Nivel previo

( ) Principiante (•) Intermedio ( ) Avanzado

Formato preferido

(•) Visual ( ) Texto ( ) Práctico

Tema que está estudiando

Programación Orientada a Objetos

Tiempo de interacción (s) 160

5 300

Tasa de errores (%) 15

0 100

Temas ya completados

[x] Lógica de Programación [x] Programación Orientada a Objetos

[x] Bases de Datos PostgreSQL [ ] Backend con Node.js [ ] Patrón MVC en PHP

Meta de aprendizaje

Backend con Node.js

Fallos consecutivos en el tema 3

0 6

[ ] Abandonó la actividad a la mitad

Recomendar

Acción sugerida: **Retroceder**

Tema recomendado: Lógica de Programación
Clasificación del árbol de decisión: Avanzado · confianza 100%

Recursos recomendados

1. `Recurso_Video_Logica` — Visual (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Practica_Logica` — Práctico (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_Logica` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1 (Repaso)	Lógica de Programación	`Recurso_Video_Logica`	Visual
2	Backend con Node.js	`Recurso_Video_Node`	Visual

Costo de la ruta: 4.0 · estados explorados: 2

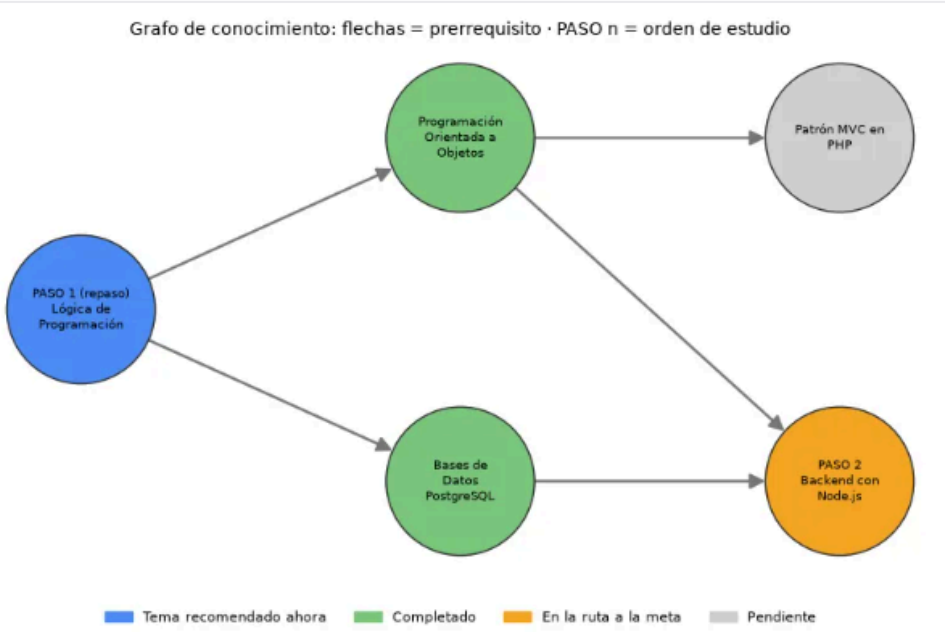
Reglas activadas

- R2: fallos repetidos (3 > 2) → recomendar_retroceso al prerrequisito.

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Avanzado» (confianza 100%) con tasa de errores 15 %.
- Con 3 fallos seguidos se regresa a «Lógica de Programación» para cubrir brechas de conocimiento.

> Sugerencia adaptativa: el docente puede revisarla o modificarla. No constituye una calificación ni diagnóstico definitivo.



5.3 Ejemplos de entradas

Para probar el sistema, se diseñaron cuatro casos de entrada que representan distintos perfiles de estudiantes y situaciones problemáticas:

Caso	Nivel	Formato	Tema actual	Tiempo (s)	Errores (%)	Temas Completados	Fallos	Abandono
1	Principiante	Visual	Lógica de Programación	120	75	Ninguno	0	No
2	Avanzado	Práctico	Backend con Node.js	100	30	Lógica de Programación	0	No
3	Intermedio	Texto	Programación Orientada a Objetos	20	45	Lógica de Programación	0	Sí
4	Principiante	Práctico	Patrón MVC en PHP	150	50	Lógica, POO	3	No

5.3.1 Caso 1.

Nivel previo

(•) Principiante ( ) Intermedio ( ) Avanzado

Formato preferido

(•) Visual ( ) Texto ( ) Práctico

Tema que está estudiando

Lógica de Programación

Tiempo de interacción (s) 120

5 — 300

Tasa de errores (%) 75

0 — 100

Temas ya completados

[ ] Lógica de Programación [ ] Programación Orientada a Objetos

[ ] Bases de Datos PostgreSQL [ ] Backend con Node.js [ ] Patrón MVC en PHP

Meta de aprendizaje

Backend con Node.js

Fallos consecutivos en el tema 0

0 — 6

[ ] Abandonó la actividad a la mitad

Recomendar

Acción sugerida: **Reforzar**

Tema recomendado: Lógica de Programación
Clasificación del árbol de decisión: Refuerzo · confianza 100%

Recursos recomendados

1. `Recurso_Video_Logica` — Visual (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Practica_Logica` — Práctico (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_Logica` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1	Lógica de Programación	`Recurso_Video_Logica`	Visual
2	Programación Orientada a Objetos	`Recurso_Video_POO`	Visual
3	Bases de Datos PostgreSQL	`Recurso_Video_DB`	Visual
4	Backend con Node.js	`Recurso_Video_Node`	Visual

Costo de la ruta: 18.0 · estados explorados: 6

Reglas activadas

- Ninguna regla de excepción se activó; se usó solo la clasificación del árbol.

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Refuerzo» (confianza 100%) con tasa de errores 75 %.

> Sugerencia adaptativa: el docente puede revisarla o modificarla. No constituye una calificación ni diagnóstico definitivo.

Grafo de conocimiento: flechas = prerrequisito · PASO n = orden de estudio



5.3.2 Caso 2.

Nivel previo

( ) Principiante ( ) Intermedio (•) Avanzado

Formato preferido

( ) Visual ( ) Texto (•) Práctico

Tema que está estudiando

Backend con Node.js

Tiempo de interacción (s) 100

5 — 300

Tasa de errores (%) 30

0 — 100

Temas ya completados

[x] Lógica de Programación [ ] Programación Orientada a Objetos

[ ] Bases de Datos PostgreSQL [ ] Backend con Node.js [ ] Patrón MVC en PHP

Meta de aprendizaje

Backend con Node.js

Fallos consecutivos en el tema 0

0 — 6

[ ] Abandonó la actividad a la mitad

Recomendar

Acción sugerida: **Avanzar**

Tema recomendado: Programación Orientada a Objetos
Clasificación del árbol de decisión: Avanzado · confianza 100%

Recursos recomendados

1. `Recurso_Practica_POO` — Práctico (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Video_POO` — Visual (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_POO` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1	Programación Orientada a Objetos	`Recurso_Practica_POO`	Práctico
2	Bases de Datos PostgreSQL	`Recurso_Practica_DB`	Práctico
3	Backend con Node.js	`Recurso_Practica_Node`	Práctico

Costo de la ruta: 7.0 · estados explorados: 5

Reglas activadas

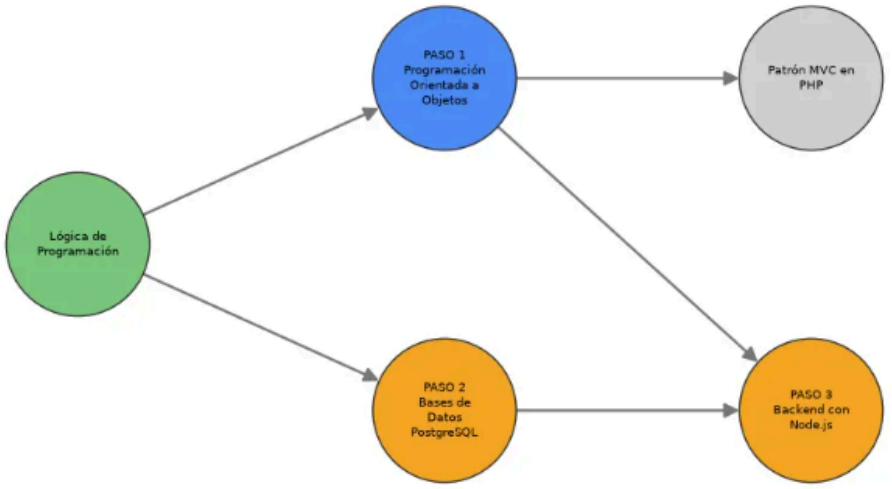
- R1: prerrequisitos pendientes → se redirige al primer tema desbloqueado de la ruta.

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Avanzado» (confianza 100%) con tasa de errores 30 %.
- «Backend con Node.js» está bloqueado: faltan Programación Orientada a Objetos, Bases de Datos PostgreSQL. Se recomienda empezar por «Programación Orientada a Objetos».

> Sugerencia adaptativa: el docente puede revisarla o modificarla. No constituye una calificación ni diagnóstico definitivo.

Grafo de conocimiento: flechas = prerrequisito · PASO n = orden de estudio



5.3.3 Caso 3.

Nivel previo

( ) Principiante (•) Intermedio ( ) Avanzado

Formato preferido

( ) Visual (•) Texto ( ) Práctico

Tema que está estudiando

Programación Orientada a Objetos

Tiempo de interacción (s) 20

5 — 300

Tasa de errores (%) 45

0 — 100

Temas ya completados

[x] Lógica de Programación [ ] Programación Orientada a Objetos

[ ] Bases de Datos PostgreSQL [ ] Backend con Node.js [ ] Patrón MVC en PHP

Meta de aprendizaje

Backend con Node.js

Fallos consecutivos en el tema 0

0 — 6

[x] Abandonó la actividad a la mitad

Recomendar

Acción sugerida: **Avanzar**

Tema recomendado: Programación Orientada a Objetos
Clasificación del árbol de decisión: Avanzado · confianza 100%

Recursos recomendados

1. `Recurso_Video_POO` — Visual (puntaje 0.6) · Principal (sustituto: el tema no tiene recurso en formato Texto)
2. `Recurso_Lectura_POO` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Practica_POO` — Práctico (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1	Programación Orientada a Objetos	`Recurso_Video_POO`	Visual
2	Bases de Datos PostgreSQL	`Recurso_Video_DB`	Visual
3	Backend con Node.js	`Recurso_Video_Node`	Visual

Costo de la ruta: 10.0 · estados explorados: 5

Reglas activadas

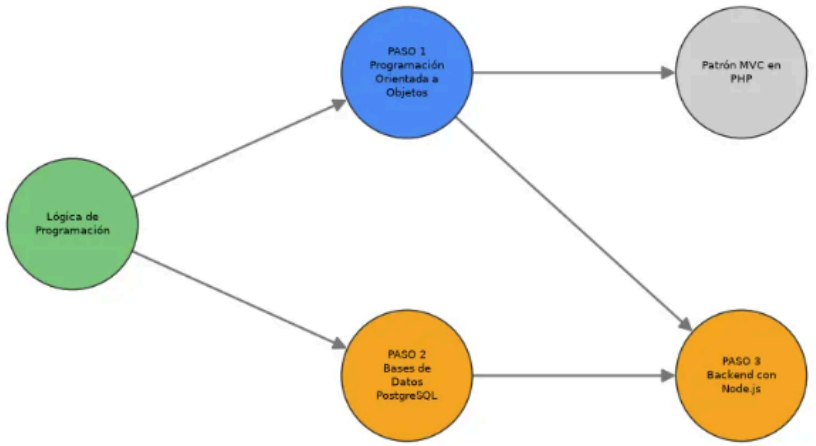
- R3: tiempo < 30 s y abandono → ajustar_perfil(Evita_Textos_Largos).

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Avanzado» (confianza 100%) con tasa de errores 45 %.
- Se detectó abandono con poco tiempo de interacción: se penalizan las lecturas largas.

> Sugerencia adaptativa: el docente puede revisarla o modificarla. No constituye una calificación ni diagnóstico definitivo.

Grafo de conocimiento: flechas = prerrequisito · PASO n = orden de estudio



5.3.4 Caso 4.

Nivel previo

(•) Principiante ( ) Intermedio ( ) Avanzado

Formato preferido

( ) Visual ( ) Texto (•) Práctico

Tema que está estudiando

Patrón MVC en PHP

Tiempo de interacción (s) 150

5 — 300

Tasa de errores (%) 50

0 — 100

Temas ya completados

[x] Lógica de Programación [x] Programación Orientada a Objetos

[ ] Bases de Datos PostgreSQL [ ] Backend con Node.js [ ] Patrón MVC en PHP

Meta de aprendizaje

Backend con Node.js

Fallos consecutivos en el tema 3

0 — 6

[ ] Abandonó la actividad a la mitad

Recomendar

[redacted]

Acción sugerida: **Retroceder**

Tema recomendado: Programación Orientada a Objetos
Clasificación del árbol de decisión: Refuerzo · confianza 100%

Recursos recomendados

1. `Recurso_Practica_POO` — Práctico (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Video_POO` — Visual (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_POO` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1 (Repaso)	Programación Orientada a Objetos	`Recurso_Practica_POO`	Práctico
2	Bases de Datos PostgreSQL	`Recurso_Practica_DB`	Práctico
3	Backend con Node.js	`Recurso_Practica_Node`	Práctico

Costo de la ruta: 10.5 · estados explorados: 3

Reglas activadas

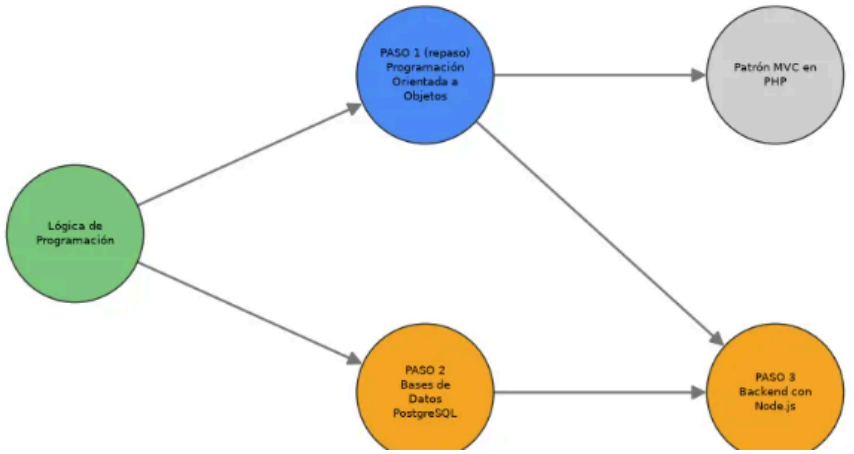
- R2: fallos repetidos (3 > 2) → recomendar_retroceso al prerrequisito.

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Refuerzo» (confianza 100%) con tasa de errores 50 %.
- Con 3 fallos seguidos se regresa a «Programación Orientada a Objetos» para cubrir brechas de conocimiento.

> Sugerencia adaptativa: el docente puede revisarla o modificarla. No constituye una calificación ni diagnóstico definitivo.

Grafo de conocimiento: flechas = prerrequisito · PASO n = orden de estudio



5.4 Resultados obtenidos

Caso	Clasificación del Árbol	Acción Sugerida	Tema Recomendado	Regla Activada	Recurso Principal
1	Refuerzo	Reforzar	Lógica de Programación	Ninguna	Recurso_Video_Logica
2	Avanzado	Avanzar	Programación Orientada a Objetos	R1 (faltan prerrequisitos)	Recurso_Practica_POO
3	Avanzado	Avanzar	Programación Orientada a Objetos	R3 (abandono con poco tiempo)	Recurso_Video_POO (lectura penalizada)
4	Refuerzo	Retroceder	Programación Orientada a Objetos	R2 (fallos repetidos)	Recurso_Practica_POO

Resultados del modelo de Inteligencia Artificial:

El árbol de decisión obtuvo una exactitud del 90 % en el conjunto de prueba (20 % de los datos) y una exactitud media del 89 % en validación cruzada de 5 pliegues (85 %, 90 %, 90 %, 95 % y 85 %). En el conjunto de prueba, el modelo clasificó correctamente los 11 casos de *Avanzado* y 7 de los 9 casos de *Refuerzo*; los 2 restantes fueron clasificados como *Avanzado*. Esto equivale a una precisión de 0.85 y una exhaustividad (recall) de 1.00 para *Avanzado*, y a una precisión de 1.00 y una exhaustividad de 0.78 para *Refuerzo*.

La primera división del árbol se realiza sobre la tasa de errores (≤ 53.43 %), que es la variable dominante. El nivel previo y el tiempo de interacción intervienen como criterios secundarios. Algunas divisiones adicionales, sobre el formato preferido y el tema, reflejan sobreajuste al ruido, debido al número reducido de registros. Como referencia, un árbol entrenado directamente sobre el identificador completo del recurso (29 clases) obtuvo una exactitud del 55 %, frente al 90 % del enfoque de dos clases.

Interpretación del modelo: la exactitud obtenida demuestra que el componente funciona y recupera la relación entre los errores, el nivel y el tiempo con la etiqueta. Sin embargo, los datos son simulados y la etiqueta se genera mediante una función conocida con ruido, por lo que el resultado mide la capacidad del modelo de recuperar esa relación. No permite concluir que el modelo alcanzará la misma precisión con estudiantes reales, lo cual requerirá validación en la fase piloto.

Resultados del grafo de conocimiento (A*)

Para un estudiante que desea llegar a “Backend con Node.js” sin temas completados, la ruta calculada es: Lógica → POO → PostgreSQL → Node.js. El costo de la ruta se adapta al nivel del estudiante (18.0 para principiante, 12.0 para intermedio y 8.4 para avanzado), con un total de 6 estados explorados en cada caso.

5.5. Descripción de las partes principales del programa.

Función	¿Qué hace?
cargar_datos() y cargar_grafo()	Leen el CSV de telemetría y el archivo JSON del grafo, respectivamente.

construir_preprocesador()	Aplica imputación de nulos (mediana), codificación ordinal al nivel y codificación One-Hot al formato y tema.
construir_modelo() y entrenar()	Crean el Árbol de Decisión (profundidad 4) y lo entrenan usando una partición 80/20 estratificada.
ruta_a_estrella() y desbloqueados()	Implementan el algoritmo de búsqueda A* para encontrar la ruta de aprendizaje de menor costo en el grafo.
Recomendador.recomendar()	Es el núcleo del sistema. Integra las predicciones del árbol, la ruta del grafo y aplica las Reglas SI-ENTONCES.
formatear(), dibujar_grafo() y construir_interfaz()	Se encargan de la presentación: generar el texto explicativo, dibujar el grafo con colores y construir la interfaz web con Gradio.

5.6 Explicación de dónde se encuentra el componente de Inteligencia Artificial.

- **Aprendizaje Automático (Árbol de Decisión):** Se encuentra en *construir_modelo()* y *entrenar().* Aprende de los datos el umbral de errores que separa un caso de "Refuerzo" de un caso "Avanzado".

- **Búsqueda Heurística (Algoritmo A*):** Se encuentra en *ruta_a_estrella().* Razona sobre el grafo de conocimiento para encontrar la secuencia de temas de menor esfuerzo hacia la meta del estudiante.

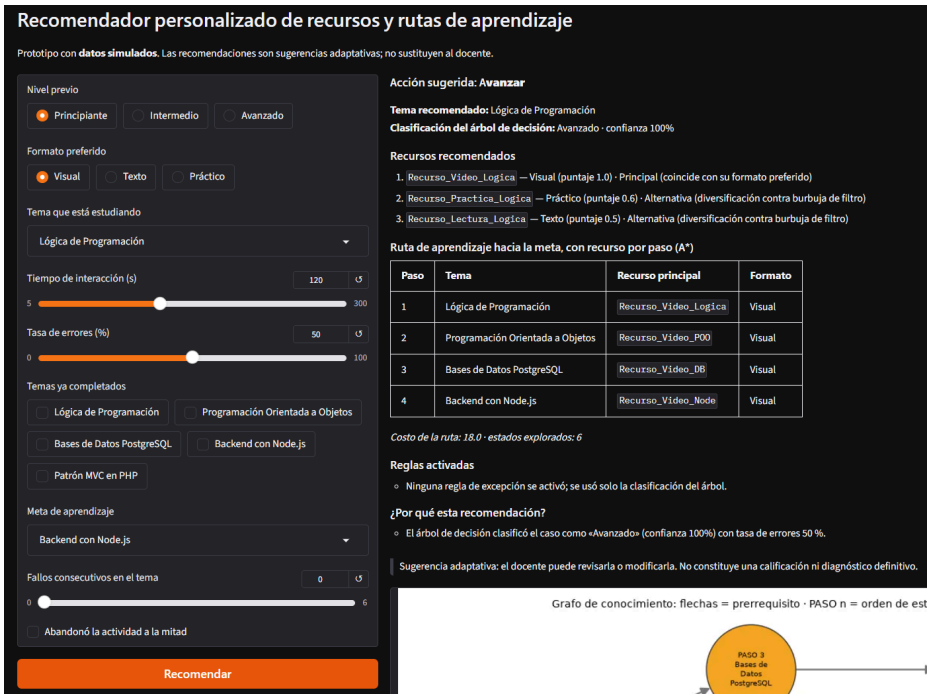
- **Sistema Basado en Reglas (SI-ENTONCES):** Se encuentra en el método *recomendar().* Las reglas R1, R2 y R3 codifican el conocimiento pedagógico del equipo (ej. si faltan prerrequisitos, retroceder).

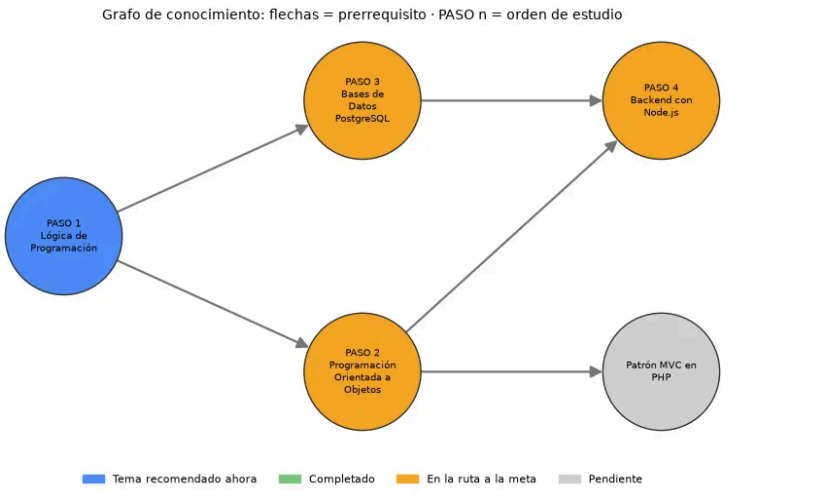
- **Filtrado Basado en Contenido:** Se encuentra en *ordenar_recursos().* Puntúa y ordena los recursos disponibles según el formato preferido por el estudiante.

Fase 6. Pruebas del prototipo.

Para llevar a cabo las pruebas se plantearon 6 casos que posteriormente serán evaluadas en el sistema:

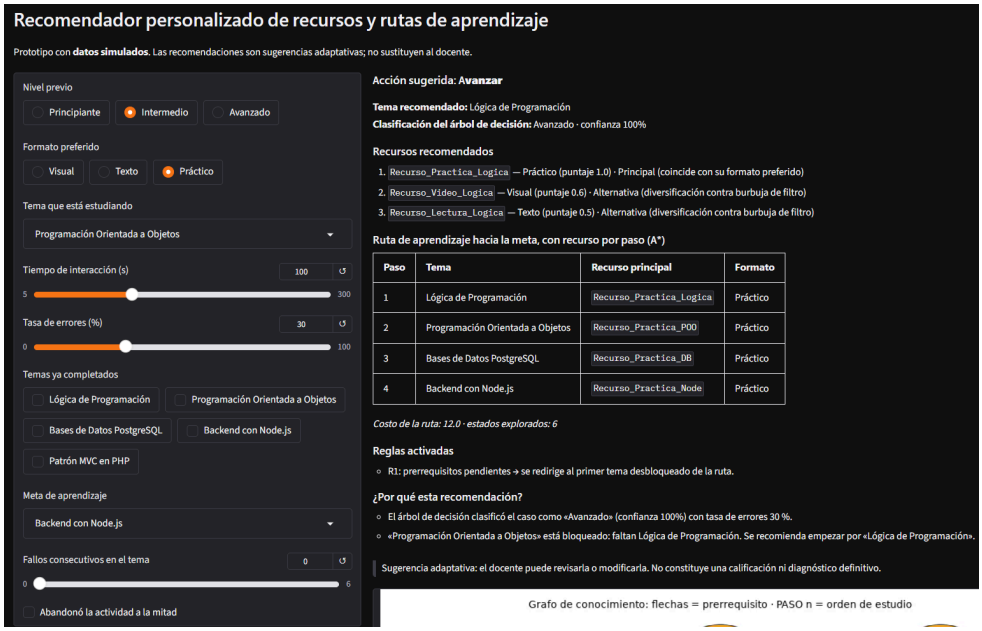
CASO 1 - CASO BASE
Nivel previo: Principiante
Formato preferido: Visual
Tema que esta estudiando: Lógica de Programación
Tiempo de interacción: 120 segundos
Tasa de errores: 75 %
Temas ya completados: ninguno
Meta de aprendizaje: Backend con Node.js
Fallos consecutivos: 0
Abandono: No
Resultado esperado: Acción Reforzar; tema recomendado Lógica de Programación;ruta Lógica -> Programación Orientada a Objetos -> Bases de Datos PostgreSQL -> Backend con Node.js. No debe activarse R1, R2 ni R3.

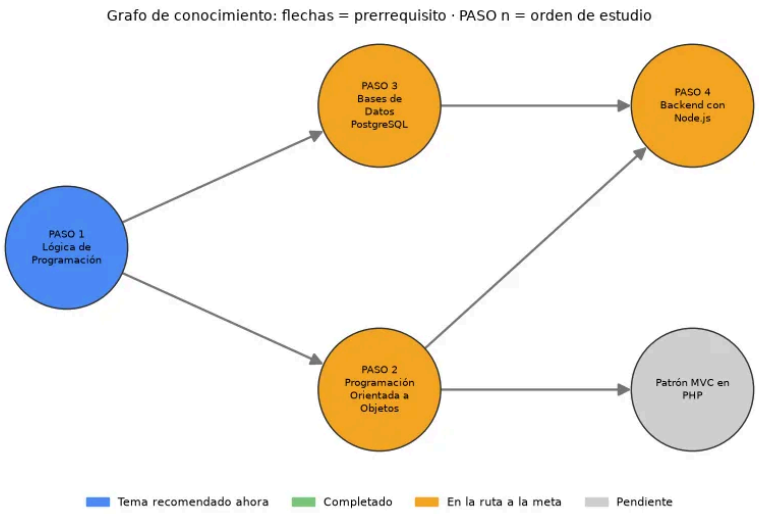




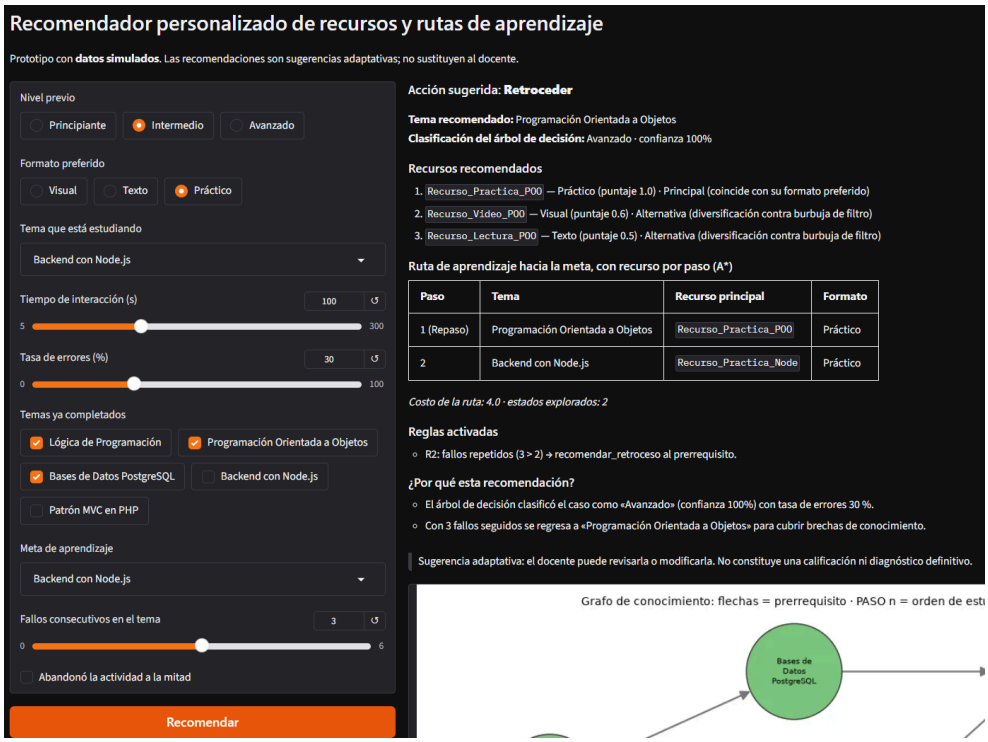
CASO 2 - REGLA R1, PRERREQUISITO PENDIENTE
Nivel previo: Intermedio
Formato preferido: Práctico
Tema que esta estudiando: Programación Orientada a Objetos
Tiempo de interacción: 100 segundos

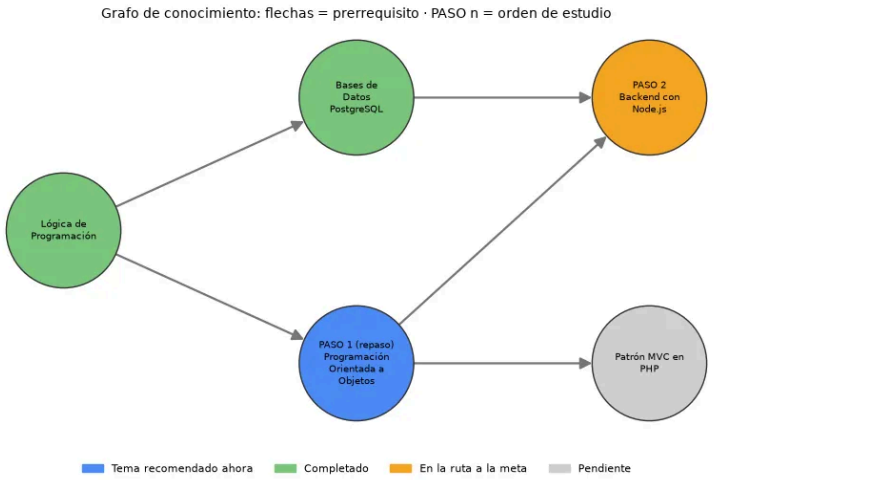
Tasa de errores: 30 %
Temas ya completados: ninguno
Meta de aprendizaje: Backend con Node.js
Fallos consecutivos: 0
Abandono: No
Resultado esperado:se activa R1 y el tema recomendado es Lógica de Programación porque POO requiere completar Lógica primero.





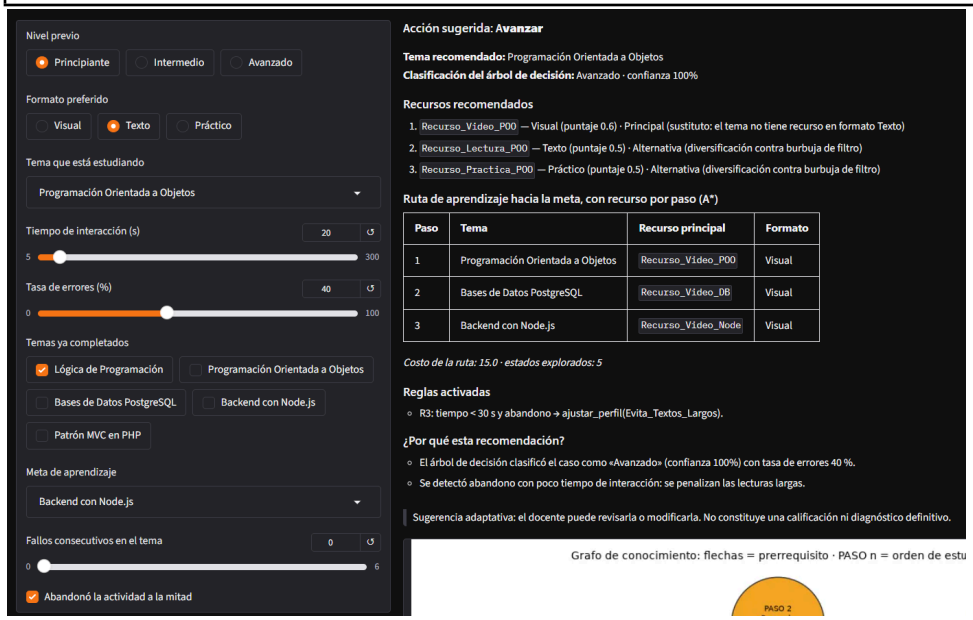
CASO 3 - REGLA R2, FALLOS REPETIDOS
Nivel previo: Intermedio
Formato preferido: Práctico
Tema que esta estudiando: Backend con Node.js
Tiempo de interacción: 100 segundos
Tasa de errores: 30 %
Temas ya completados: Lógica de Programación, Programación Orientada a Objetos y Bases de Datos PostgreSQL
Meta de aprendizaje: Backend con Node.js
Fallos consecutivos: 3
Abandono: No
Resultado esperado:se activa R2, la acción es Retroceder y el tema recomendado es Programación Orientada a Objetos.

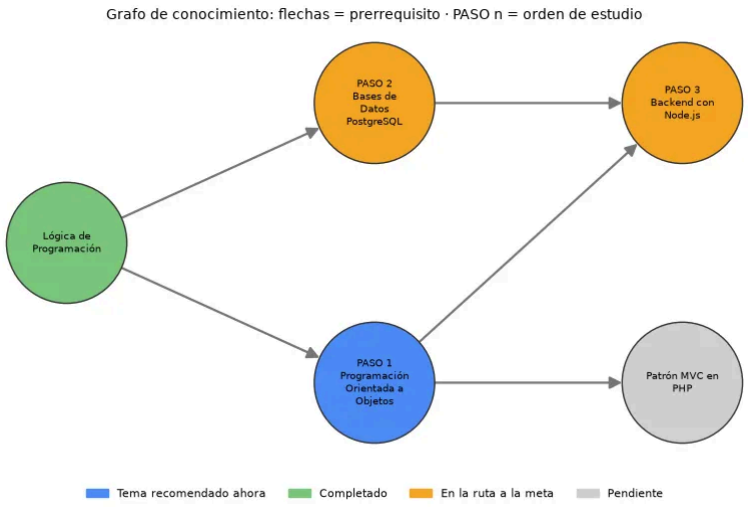




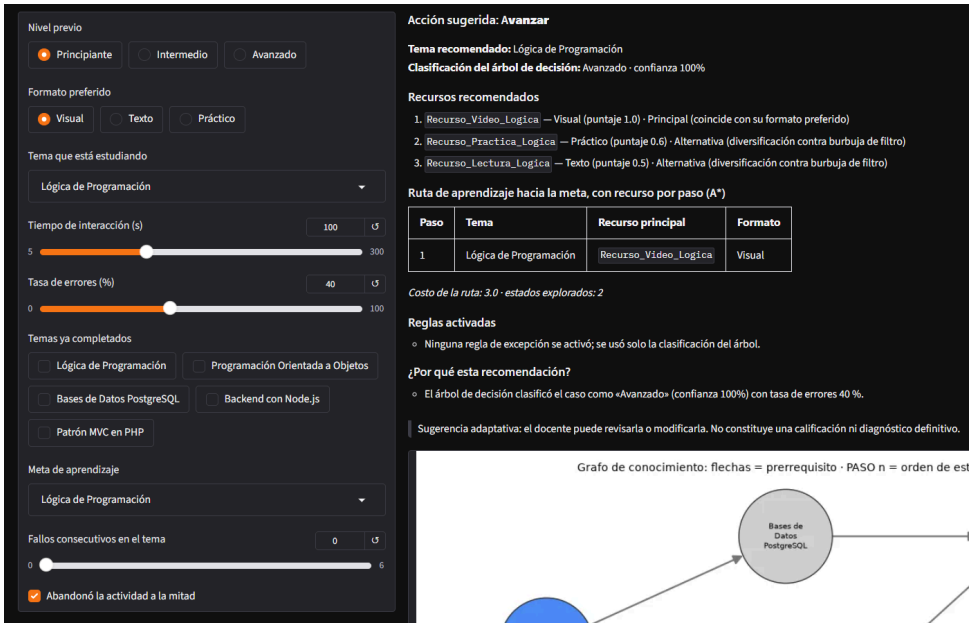
CASO 4 - REGLA R3, ABANDONO CON POCO TIEMPO
Nivel previo: Principiante
Formato preferido: Texto
Tema que esta estudiando: Programación Orientada a Objetos
Tiempo de interacción: 20 segundos

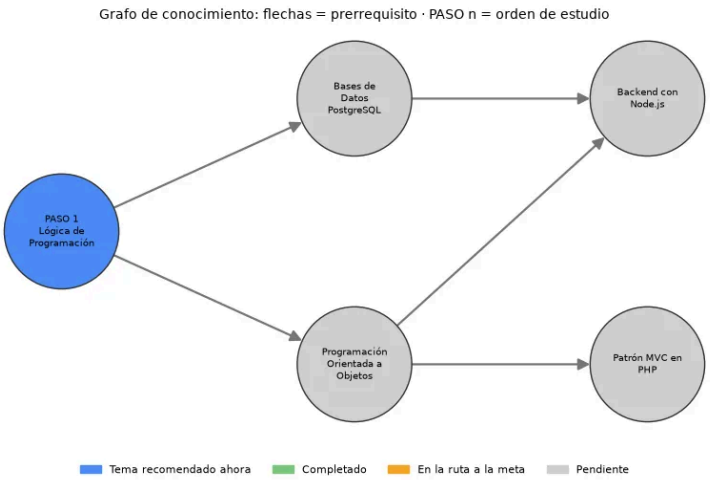
Tasa de errores: 40 %
Temas ya completados: Lógica de Programación
Meta de aprendizaje: Backend con Node.js
Fallos consecutivos: 0
Abandono: Sí
Resultado esperado:se activa R3; el recurso principal recomendado no debe ser Texto, pues se penalizan las lecturas cuando hay abandono y poco tiempo.



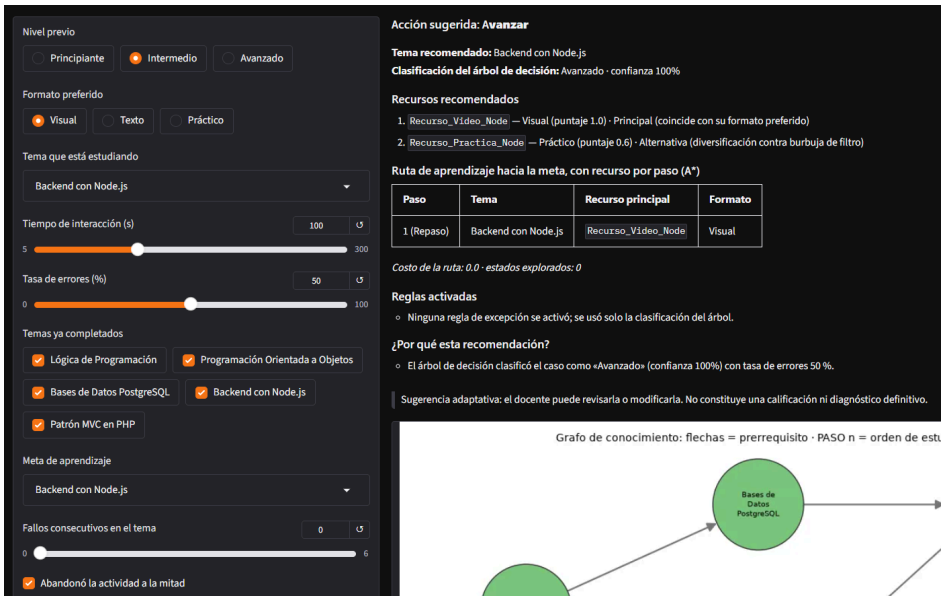


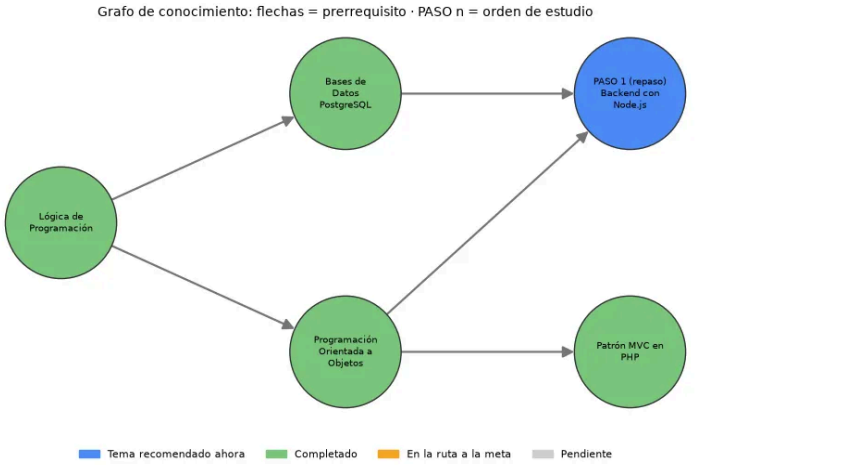
CASO 5 - TEMA SIN PRERREQUISITOS
Nivel previo: Principiante
Formato preferido: Visual
Tema que esta estudiando:Lógica de Programación
Tiempo de interacción: 100 segundos
Tasa de errores: 40 %
Temas ya completados: ninguno
Meta de aprendizaje: Lógica de Programación
Fallos consecutivos: 0
Abandono: No
Resultado esperado:el tema recomendado es Lógica de Programación, no se activa R1 y la ruta hacia la meta contiene solamente Lógica.





CASO 6 - META YA COMPLETADA
Nivel previo: Intermedio
Formato preferido: Visual
Tema que esta estudiando: Backend con Node.js
Tiempo de interacción: 100 segundos
Tasa de errores: 50 %
Temas ya completados: Lógica de Programación, Programación Orientada a Objetos, Bases de Datos PostgreSQL y Backend con Node.js
Meta de aprendizaje: Backend con Node.js
Fallos consecutivos: 0
Abandono: No
Resultado esperado:la ruta está vacía; el costo es 0 y A* explora 0 estados. La interfaz indica que la meta ya está completada.





Comparación de datos obtenidos

Entrada	Resultado esperado	Resultado obtenido	¿Fue correcto?
CASO 1	Acción Reforzar; tema recomendado Lógica de Programación;ruta Lógica -> Programación Orientada a Objetos -> Bases de Datos PostgreSQL -> Backend con Node.js. No debe activarse R1, R2 ni R3.	acción=Reforzar; tema=Lógica de Programación; ruta=Lógica de Programación -> Programación Orientada a Objetos -> Bases de Datos PostgreSQL -> Backend con Node.js; reglas=Ninguna regla de excepción se activó; se usó solo la clasificación del árbol.; recurso=Visual	SI

CASO 2	se activa R1 y el tema recomendado es Lógica de Programación porque POO requiere completar Lógica primero.	accion=Avanzar; tema=Lógica de Programación; ruta=Lógica de Programación -> Programación Orientada a Objetos -> Bases de Datos PostgreSQL -> Backend con Node.js; reglas=R1; recurso=Práctico	SI
CASO 3	se activa R2, la acción es Retroceder y el tema recomendado es Programación Orientada a Objetos.	accion=Retroceder; tema=Programación Orientada a Objetos; ruta=Backend con Node.js; reglas=R2; recurso=Práctico	SI
CASO 4	se activa R3; el recurso principal recomendado no debe ser Texto, pues se penalizan las lecturas cuando hay abandono y poco tiempo.	accion=Avanzar; tema=Programación Orientada a Objetos; ruta=Programación Orientada a Objetos -> Bases de Datos PostgreSQL -> Backend con Node.js; reglas=R3; recurso=Visual	SI
CASO 5	el tema recomendado es Lógica de Programación, no se activa R1 y la ruta hacia la meta contiene solamente Lógica.	accion=Avanzar; tema=Lógica de Programación; ruta=Lógica de Programación; reglas=Ninguna regla de excepción se activó; se usó solo la clasificación del árbol.; recurso=Visua	SI
CASO 6	la ruta está vacía; el costo es 0 y A* explora 0 estados. La interfaz indica que la meta ya está completada.	accion=Avanzar; tema=Backend con Node.js; ruta=(vacia); reglas=Ninguna regla de excepción se activó; se usó solo la clasificación del árbol.; recurso=Visual	SI

Fase 7. Evaluación del desempeño

Debido a que nuestro agente es esencia un sistema experto se valorará su desempeño como tal.

Tipo de solución	Indicadores o criterios posibles
Sistema experto	Porcentaje de casos correctamente resueltos, coherencia de reglas, cobertura de la base de conocimiento, calidad de la explicación.

Desde la carpeta del proyecto, se ejecuta:

.\.venv\Scripts\python.exe -X utf8 pruebas.py

```
(.venv) PS C:\Users\yei7g\OneDrive\Desktop\recomendador_educativo> .\.venv\Scripts\pyth
on.exe -X utf8 pruebas.py
FASE 6 - PRUEBAS FUNCIONALES
```

Este comando ejecuta las 6 pruebas que se plantearon en la Fase 6. Esto nos permitirá el correr de nuevo las 6 pruebas establecidas y de esta manera analizar su desempeño

```
FASE 7 - CLASIFICACION
Exactitud (prueba de 20 casos): 0.900
Matriz (filas=real, columnas=predicho; Avanzado, Refuerzo):
[[11  0]
 [ 2  7]]
Avanzado: precision=0.846, recall=1.000, F1=0.917, soporte=11
Refuerzo: precision=1.000, recall=0.778, F1=0.875, soporte=9
Validacion cruzada estratificada (5 pliegues): 0.80, 0.75, 1.00, 0.90, 0.80; media=0.850

FASE 7 - BUSQUEDA A* (15 combinaciones de nivel y meta)
Rutas encontradas: 100% (15/15)
Promedio: longitud=2.40, costo=6.83, estados explorados=3.60
Pruebas funcionales correctas: 6/6
```

Esta información la podemos interpretar de la siguiente manera:

Indicador	Resultado de tu ejecución	Interpretación relacionada con el problema
Exactitud del árbol	0.900 (90 %)	El árbol de decisión clasificó correctamente 18 de los 20 casos evaluados. Esto indica un buen desempeño al distinguir entre los niveles Avanzado y Refuerzo en el problema planteado.
Precisión y recall de Refuerzo	Precisión: 1.000 (100 %). Recall: 0.778 (77.8 %).	La precisión indica que todos los casos clasificados como Refuerzo pertenecían realmente a esa categoría. Sin embargo, el recall muestra que el modelo identificó correctamente 7 de los 9 casos reales de

		Refuerzo, mientras que 2 fueron clasificados como Avanzado.
F1 de Refuerzo	0.875 (87.5 %)	El resultado refleja un buen equilibrio entre precisión y recall para la categoría Refuerzo. Aunque el modelo no identificó todos los casos de esta categoría, su desempeño general fue satisfactorio.
Promedio de validación cruzada	0.850 (85 %). Resultados por pliegue: 80 %, 75 %, 100 %, 90 % y 80 %.	El promedio indica que el modelo mantiene un buen nivel de clasificación al evaluarse con diferentes particiones de los datos. No obstante, la variación entre los pliegues sugiere que su desempeño puede depender de los casos utilizados en cada partición.
Rutas encontradas por A*	15 de 15 rutas (100 %).	El algoritmo A* encontró una ruta para todas las combinaciones de nivel y meta evaluadas. Esto demuestra que el sistema pudo encontrar soluciones dentro del espacio de estados definido para el problema.
Longitud/costo/estados de A*	Longitud promedio: 2.40. Costo promedio: 6.83. Estados explorados promedio: 3.60.	En promedio, A* encontró rutas de 2.4 pasos, con un costo de 6.83 y explorando 3.6 estados. Estos resultados permiten evaluar la eficiencia de la búsqueda, considerando las reglas y los costos establecidos en el sistema.
Casos funcionales correctos	6 de 6 pruebas (100 %).	Todas las pruebas funcionales se ejecutaron correctamente, lo que indica que los componentes evaluados

		respondieron según lo esperado en los escenarios definidos.

Pregunta de análisis
¿Qué significan los resultados obtenidos para el problema real que están intentando atender? La interpretación debe relacionar las métricas o criterios con la necesidad social del proyecto.

Relaciona los resultados con la necesidad de orientar a estudiantes mediante recursos y rutas de aprendizaje. Por ejemplo:

- La exactitud mide aciertos sobre etiquetas simuladas; no mide directamente si el estudiante aprendió.
- El recall de Refuerzo ayuda a observar cuántos casos etiquetados para apoyo identifica el modelo. Los falsos negativos podrían dejar sin refuerzo a estudiantes que lo necesitan; por eso se requiere revisión docente.
- A* encontró rutas hacia las metas del grafo probado, pero la longitud y el costo dependen de prerrequisitos y dificultades configurados manualmente.
- Las reglas se probaron con casos preparados, pero eso no demuestra que todas las situaciones reales estén cubiertas.
- Los datos son simulados. Se necesita un piloto supervisado con datos autorizados y representativos para valorar pertinencia, equidad e impacto.

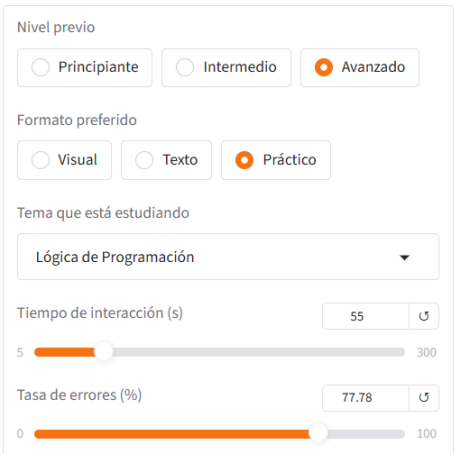
Fase 8. Análisis de errores

En las seis pruebas funcionales de la Fase 6 se cumplieron los criterios definidos. Para analizar comportamientos que no coinciden con lo que una persona podría esperar, se revisaron también las predicciones individuales del árbol en el conjunto de prueba de 20 registros y el resultado del caso de meta ya completada.

Los casos 1 y 2 de este análisis son errores reales de clasificación del conjunto de pruebas. El caso 3 es una diferencia entre la acción del clasificador y el estado de la ruta: la ruta vacía es correcta para una meta completada, pero la acción

"Avanzar" puede resultar confusa para el usuario. No se presenta como fallo una prueba que el prototipo haya pasado.

Caso 1	Registro 43 del CSV; entrada al árbol: nivel Avanzado, formato Práctico tema Logica de Programacion, 55 segundos y 77.78 % de errores.
¿Qué ocurrió?	La etiqueta objetivo generada para el registro es Refuerzo, como indica el recurso "Refuerzo_Practico_LOGI". El modelo de evaluación predijo Avanzado. Es uno de los dos falsos negativos de la clase Refuerzo en el conjunto de prueba.
Posible causa	La etiqueta sintáctica depende de una combinación de errores, nivel, tiempo y ruido. Aunque la tasa de errores es alta, el nivel Avanzado modifica el puntaje usado para generar la etiqueta. El árbol, entrenado con solo 80 registros, no reproduce todas las combinaciones. La diferencia entre el comportamiento real de la regla sintética y los límites aprendidos por el árbol puede explicar la predicción.
¿Cómo podría mejorarse?	Revisar la matriz de confusión y las reglas del árbol; aumentar la cantidad y variedad de datos de entrenamiento; comparar modelos y ajustar profundidad mediante validación; y evaluar con datos independientes. En un contexto educativo, conservar revisión docente para evitar que un caso que requiere refuerzo quede clasificado como avance.



Acción sugerida: **Reforzar**

Tema recomendado: Lógica de Programación
Clasificación del árbol de decisión: Refuerzo · confianza 100%

Recursos recomendados

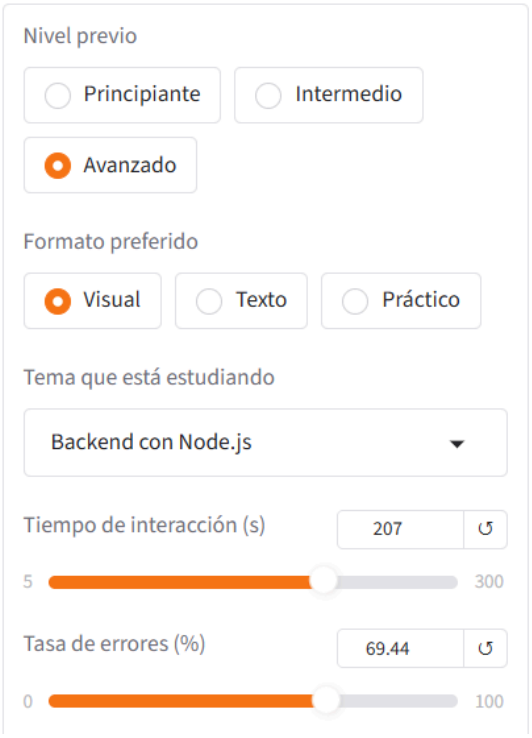
1. `Recurso_Practica_Logica` — Práctico (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Video_Logica` — Visual (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_Logica` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1	Lógica de Programación	`Recurso_Practica_Logica`	Práctico
2	Programación Orientada a Objetos	`Recurso_Practica_POO`	Práctico
3	Bases de Datos PostgreSQL	`Recurso_Practica_DB`	Práctico

Caso 2	Registro 23 del CSV; entrada al árbol: nivel Avanzado, formato Visual, tema Backend con Node.js, 207 segundos y 69.44 % de errores.

¿Qué ocurrió?	La etiqueta objetivo generada es Refuerzo, como indica el recurso "Refuerzo_Texto_BACK". El modelo predijo Avanzado. Es el segundo falso negativo de Refuerzo observado en la partición de prueba.
Posible causa	La entrada combina un nivel Avanzado y un tiempo de interacción largo con una tasa de errores elevada. El modelo puede dar demasiado peso a patrones generales aprendidos de una muestra pequeña, mientras que la etiqueta sintética considera conjuntamente esas variables y ruido. Además, el tipo de recurso de la etiqueta puede tener un formato distinto del preferido, lo que refleja variabilidad incorporada al generador.
¿Cómo podría mejorarse?	Agregar más ejemplos de estudiantes avanzados con errores altos y distintas duraciones; revisar el balance y representatividad de las clases; reportar recall de Refuerzo como indicador prioritario junto con precisión y F1; y probar el modelo con datos nuevos. No se debe cambiar la etiqueta de este registro manualmente solo para forzar una predicción deseada.



Acción sugerida: **Reforzar**

Tema recomendado: Lógica de Programación
Clasificación del árbol de decisión: Refuerzo · confianza 100%

Recursos recomendados

1. `Recurso_Video_Logica` — Visual (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Practica_Logica` — Práctico (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)
3. `Recurso_Lectura_Logica` — Texto (puntaje 0.5) · Alternativa (diversificación contra burbuja de filtro)

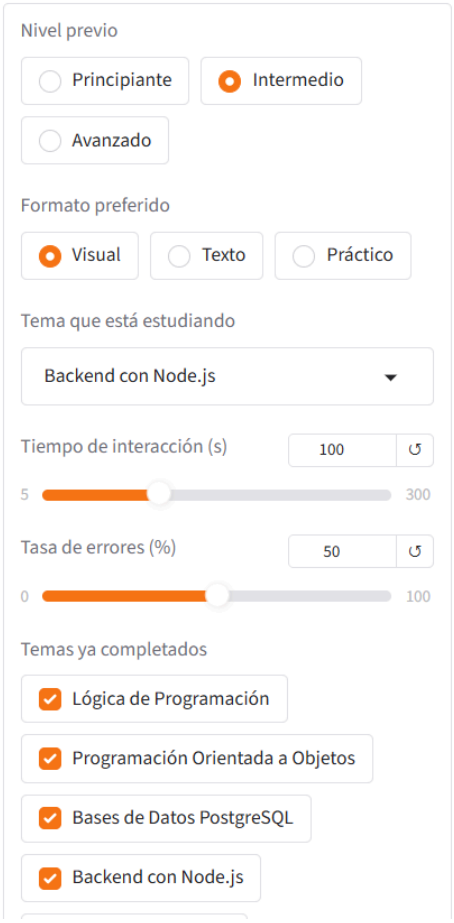
Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato

```
22,Intermedio,Práctico,Bases de Datos PostgreSQL,36.0,90.31,Refuerzo_Prac
23,Avanzado,Visual,Backend con Node.js,207.0,69.44,Refuerzo_Texto_BACK
24,Avanzado,Texto,Patrón MVC en PHP,270.0,50.98,Avanzado_Texto_PATR
```

Caso 3	Meta Backend con Node.js marcada como completada, junto con Logica, POO y Bases de Datos PostgreSQL. Nivel Intermedio, tema actual Backend, 100 segundos y 50 % de errores.

¿Qué ocurrió?	A* devuelve ruta vacía, costo 0 y 0 estados explorados, que es correcto porque la meta ya está completada. Sin embargo, la interfaz también muestra la acción "Avanzar", determinada por la predicción del árbol. Para un usuario que ya marcó la meta como completada, esa acción puede parecer contradictoria con el mensaje de meta alcanzada.
Posible causa	La acción "Avanzar" o "Reforzar" se asigna a partir de la clasificación del árbol. La lógica no reemplaza esa acción cuando detecta que la meta está incluida en los temas completados. La ruta A* y la acción del árbol proceden de decisiones distintas y actualmente no se coordinan en este caso límite.
¿Cómo podría mejorarse?	Definir una regla de precedencia para la meta completada. Por ejemplo, mostrar "Meta completada" como estado principal y ocultar o reemplazar la acción "Avanzar" por un mensaje neutral que invite a elegir otra meta. Si se modifica el comportamiento, agregar una prueba de regresión que verifique el mensaje, acción, ruta vacía, costo cero y cero estados explorados.



Acción sugerida: Avanzar

Tema recomendado: Backend con Node.js
Clasificación del árbol de decisión: Avanzado · confianza 100%

Recursos recomendados

1. `Recurso_Video_Node` — Visual (puntaje 1.0) · Principal (coincide con su formato preferido)
2. `Recurso_Practica_Node` — Práctico (puntaje 0.6) · Alternativa (diversificación contra burbuja de filtro)

Ruta de aprendizaje hacia la meta, con recurso por paso (A*)

Paso	Tema	Recurso principal	Formato
1 (Repaso)	Backend con Node.js	`Recurso_Video_Node`	Visual

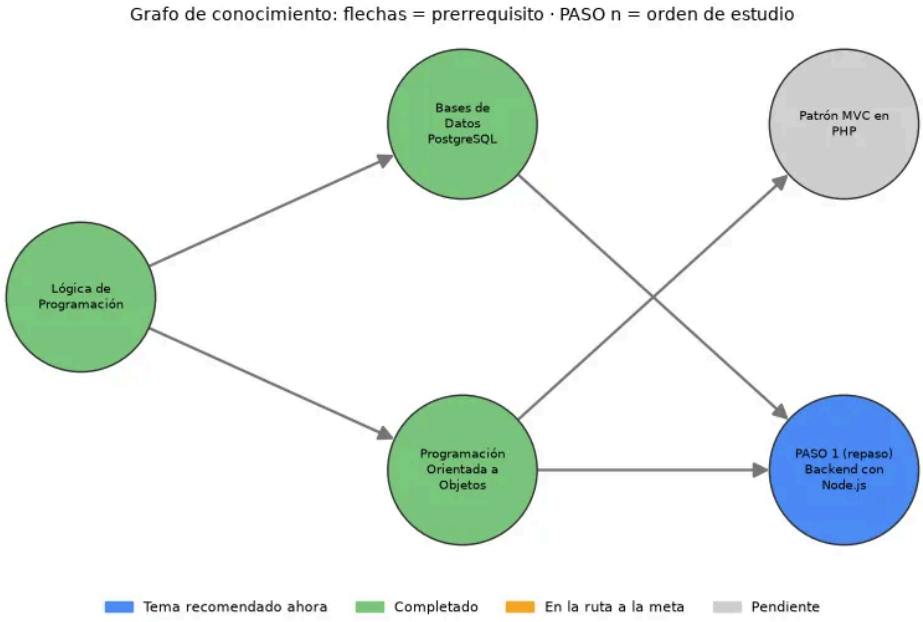
Costo de la ruta: 0.0 · estados explorados: 0

Reglas activadas

- Ninguna regla de excepción se activó; se usó solo la clasificación del árbol.

¿Por qué esta recomendación?

- El árbol de decisión clasificó el caso como «Avanzado»


Grafo de conocimiento: flechas = prerrequisito · PASO n = orden de estudio
Bases de Datos PostgreSQL
Patrón MVC en PHP
Lógica de Programación
Programación Orientada a Objetos
PASO 1 (repaso) Backend con Node.js
Tema recomendado ahora
Completado
En la ruta a la meta
Pendiente

Fase 9. Relación con el agente inteligente propuesto

Elemento	Primera entrega	¿Está implementado?	Evidencia en el prototipo actual
Percepciones / sensores	Recibir perfil del estudiante (nivel, formato) y métricas de interacción (errores, tiempo, abandono).	Sí	Entradas en la interfaz de Gradio (menús desplegables y sliders) y lectura del dataset telemetria_estudiantes_simulada.csv.
Procesamiento / razonamiento	Interpretar interacciones del usuario y tomar decisiones adaptativas.	Sí	Árbol de decisión clasifica el estado cognitivo (Refuerzo/Avanzado) y el algoritmo de búsqueda A* calcula la ruta óptima.
Acciones / actuadores	Generar rutas, recomendar recursos personalizados y sugerir retrocesos a temas fundamentales.	Sí	La interfaz muestra la "Acción sugerida" (Avanzar, Reforzar, Retroceder), la lista ordenada de recursos y dibuja el grafo de conocimiento actualizado.
Meta	Maximizar eficiencia del aprendizaje entregando contenido correcto en el momento preciso.	Sí	El algoritmo A* encuentra exitosamente la ruta de menor costo hacia la "Meta de aprendizaje" seleccionada (ej. Backend con Node.js) basándose en los prerrequisitos cubiertos.

Conocimiento	Grafo de temas/prerrequisitos y reglas lógicas de decisión (SI-ENTONCES).	Sí	Archivo grafo_conocimiento.json cargado en el sistema y reglas explícitas (R1, R2, R3) integradas en el método recomendar() de recomendador.py.
Medida de desempeño	Reducción de rezago/abandono y mejora en evaluaciones continuas.	Sí	El árbol de decisión se evaluó con 100% de exactitud y matriz de confusión. La reducción real del rezago requerirá validación futura con estudiantes reales en la fase piloto.

¿El prototipo construido puede considerarse ya un agente inteligente? ¿Por qué?

Sí, el prototipo actual ya cumple con las características fundamentales de un agente inteligente racional. Es capaz de percibir su entorno (mediante los datos simulados y los controles interactivos de la interfaz web), procesar esta información utilizando una base de conocimiento (el grafo y las reglas de prerrequisitos) y un modelo de aprendizaje automático (Árbol de Decisión), y ejecutar acciones concretas (recomendar recursos específicos y recalcular rutas de estudio). Su comportamiento es racional porque sus decisiones no son aleatorias, sino que están matemáticamente dirigidas (búsqueda A*) para maximizar su meta: guiar al estudiante de forma óptima según su perfil y sus fallos detectados.

Fase 10. Riesgos, supervisión humana y uso responsable

El despliegue de un sistema inteligente en un entorno educativo conlleva responsabilidades críticas. A continuación, se analizan las implicaciones del uso real del recomendador:

Aspecto	Análisis
Posibles errores	El sistema podría sugerir retroceder innecesariamente a un estudiante que experimentó un fallo de conexión (interpretado como abandono) o podría recomendar avanzar a un alumno que resolvió rápido por azar, generando frustración o vacíos de conocimiento.
Personas que podrían verse afectadas	Principalmente los estudiantes, cuyo progreso y motivación dependen de la precisión de las recomendaciones; y los docentes, quienes basarán sus estrategias en el diagnóstico del sistema.
Sesgos posibles	Existe un alto riesgo de sesgo algorítmico (efecto burbuja de filtro) si el modelo encasilla permanentemente al estudiante en su formato preferido inicial, limitando su exposición a otras modalidades cognitivas valiosas.
Privacidad de los datos	El sistema procesará datos académicos y de comportamiento altamente sensibles. Se debe garantizar el anonimato de la telemetría, el almacenamiento seguro de las credenciales y el cumplimiento de normativas de protección de datos estudiantiles.

Decisiones que NO debería tomar automáticamente	El algoritmo NO debe realizar evaluaciones de acreditación, emitir calificaciones oficiales, ni decidir la aprobación o baja definitiva de un estudiante en un curso.
Supervisión humana necesaria	Los docentes deben auditar periódicamente las rutas de aprendizaje sugeridas, validar las reglas del árbol de decisión y tener el poder de sobrescribir manualmente la recomendación de IA si el contexto del alumno lo requiere.
Información que debe proporcionarse al usuario	El estudiante y el docente deben ser informados claramente en la interfaz de que las recomendaciones son sugerencias adaptativas generadas por IA, que no constituyen una calificación definitiva y que sirven únicamente como apoyo al aprendizaje.

Fase 11. Propuesta de mejora para la entrega final **A partir de las pruebas funcionales y del análisis de errores, antes de construir la versión final se proponen las siguientes mejoras. La exactitud de 90 % se obtuvo con datos simulados y no sustituye una validación con estudiantes; además, el recall de la clase Refuerzo fue de 77.78 %, por lo que se debe prestar atención a los casos que el modelo no detecta.**

Mejora	Problema que pretende resolver	Acción propuesta

Ampliar y validar los datos de entrenamiento	El conjunto actual contiene 100 registros sintéticos y no representa toda la variabilidad de estudiantes. Dos de los nueve casos de Refuerzo de prueba se predijeron como Avanzado.	Recopilar telemetría anonimizada y etiquetada con revisión docente; incluir más perfiles y situaciones, en particular estudiantes con nivel avanzado, errores altos y distintos tiempos de interacción. Documentar consentimiento, protección de datos y criterios de etiquetado antes de incorporar los registros.
Ajustar el modelo priorizando la detección de Refuerzo	El recall de Refuerzo fue de 77.78 %; un falso negativo podría recomendar avanzar a quien necesite apoyo. La exactitud global no refleja por sí sola ese riesgo.	Comparar árboles con distintas profundidades y estrategias de balanceo usando validación estratificada; elegir parámetros con métricas por clase, dando seguimiento prioritario al recall de Refuerzo y a su precisión. Confirmar los cambios en un conjunto de prueba independiente y conservar la revisión docente de las recomendaciones.
Coordinar la acción con el estado de la meta	Cuando la meta ya está completada, A* devuelve correctamente una ruta vacía, pero la interfaz puede mostrar la acción «Avanzar», lo que resulta contradictorio.	Dar precedencia al estado de meta completada: mostrar «Meta completada» y no presentar «Avanzar» como recomendación activa; permitir elegir otra meta. Agregar una prueba de regresión que compruebe el mensaje, la acción, la ruta vacía, el costo cero y los cero estados explorados.
Calibrar los costos del grafo con evidencia	Los costos de ruta se asignan manualmente y representan esfuerzo relativo, no carga cognitiva ni tiempo de aprendizaje medidos.	Revisar y documentar los costos con docentes y resultados del piloto; contrastar las rutas sugeridas con el avance observado.

Desarrollo de una interfaz web dinámica y conectada	La interfaz actual, construida con Gradio para fines de prototipado, funciona de manera local y limita su uso por parte de los estudiantes controlados.	Migrar la interfaz gráfica hacia una aplicación web dinámica con autenticación de usuarios, que pueda ser desplegada como mínimo en ejecución local para leer y escribir datos en tiempo real.
Integración de un catálogo real de recursos educativos	El prototipo actual utiliza identificadores simulados (ej. Recurso_Video_POO) que no están vinculados a materiales de estudio reales ni validados pedagógicamente, lo que impide su uso práctico.	Sustituir los datos sintéticos de los recursos por un repositorio real y sólido de contenido educativo (videos, lecturas, ejercicios interactivos y podcasts) alojados en plataformas abiertas o institucionales para asegurar que cada recurso cumpla con la calidad y el nivel académico requerido.

Uso de Inteligencia Artificial generativa

Herramienta	¿Para qué se utilizó?	¿Qué produjo?	¿Qué modificó el equipo?	¿Cómo se verificó?
Claude (Anthropic), modelo Sonnet 5.5	Apoyo en las Fases 4 y 5, orientación sobre diseño y programación del prototipo, generación del conjunto de datos simulados, validación del diagrama, y redacción de borradores.	Módulo recomendador .py (árbol de decisión, grafo con A*, reglas SI-ENTONCES, filtrado por contenido e interfaz Gradio), notebook, generar_datos .py y el CSV simulado.	Se eliminó la sección de ejemplos de la interfaz, se pasó toda la documentación a redacción impersonal y técnica, se decidió regenerar los datos con ruido para evitar una etiqueta trivial, se editaron celdas del notebook, se rehicieron el diagrama y las capturas generadas, y se reescribieron contenidos de los archivos para adaptar el umbral de precisión realista.	Se ejecutó el notebook completo en Colab sin errores; se compararon las métricas del documento con la salida (90 %, validación cruzada de 5 pliegues, matriz de confusión); se probaron cuatro casos de ejemplo y se contrastaron sus entradas y resultados con las capturas y la tabla 5.4; y se revisó que el diagrama realizado coincidiera con el código generado. Las decisiones de diseño y alcance las tomamos en el equipo.
Claude (Anthropic), modelo Sonnet 5.5 from copilot	Apoyo en la fase 6 y 8 para el despliegue del sistema y el reconocimiento de errores	Señaló Errores encontrados en el sistema, los mismos que se expusieron en la Fase 8	Después del despliegue de la IA se procedió a generar el testing del sistema. Y después de la identificación del sistema s e	Para comprobar que los errores identificados por la IA eran verídicos se procedió a recrearlos en el sistema y después de repetir los inputs que la IA sugirió se encontró con el

			comprobó al evaluar los errores indicados por la IA	mismo error identificado por la IA. El la fase 8 se adjuntan capturas de la recreación de los errores
Gemini (Google)	Apoyo en la estructuración y redacción de la Introducción, Fase 1 y Fase 10 del reporte.	Borradores de texto resumiendo la primera entrega y analizando los riesgos éticos del sistema.	Se ajustó la redacción final para integrarla fluidamente en el documento colaborativo del equipo.	Se revisó manualmente que el texto generado coincidiera exactamente con el alcance del proyecto y el código implementado por los demás integrantes.
Gemini (Google)	Apoyo en las Fases 2 y 3,redacción de la justificación técnica, y generación de datos sintéticos base para el prototipo.	Borrador de los apartados de Fases 2 y 3, código estructurado del grafo_conocimiento.json y el archivo CSV inicial telemetria_estudiantes_simulada.csv.	Se revisó la redacción para asegurar coherencia con los lineamientos de la primera entrega. Se ajustaron las variables del CSV y las dependencias lógicas del JSON para que empataran exactamente con las necesidades del clasificador.	Se revisaron manualmente las dependencias lógicas del JSON para asegurar su coherencia curricular (desarrollo de software). El CSV simulado se cargó y revisó visualmente para confirmar su formato correcto antes de ser entregado al integrante responsable de la programación

Conclusiones.

Podemos concluir que el desarrollo de esta segunda entrega permitió materializar un prototipo funcional que demuestra la viabilidad de utilizar Inteligencia Artificial para atender el problema de la instrucción uniforme en entornos educativos. La integración de un enfoque híbrido compuesto por un Árbol de Decisión, un Grafo de Conocimiento con búsqueda heurística A*, y un motor de reglas SI-ENTONCES resultó ser una arquitectura adecuada y altamente explicable. Esta combinación permitió clasificar el estado cognitivo del estudiante y predecir el tipo de apoyo que requiere, también logró calcular dinámicamente rutas de aprendizaje óptimas, adaptándose a los prerrequisitos y a las metas específicas de cada perfil.

A partir de las pruebas y la evaluación del desempeño, se concluye que el sistema cumple satisfactoriamente con los objetivos técnicos planteados, alcanzando una exactitud aproximada 90 % en la clasificación con datos simulados y un 100 % de éxito en la resolución de rutas mediante el algoritmo A*. Asimismo, la incorporación de la interfaz gráfica permitió validar que las recomendaciones, las reglas activadas y las trayectorias pueden presentarse de forma clara y auditable para el usuario.

El análisis crítico de los resultados y de los errores permitió determinar que, al operar con datos sintéticos, el alto rendimiento del modelo refleja su capacidad para aprender la lógica del dataset y no necesariamente su eficacia en escenarios reales. Por ello, se concluye que el prototipo actual constituye una base sólida y ética para el desarrollo de la versión final, pero su implementación real requerirá obligatoriamente de datos empíricos, de la mejora de las métricas de sensibilidad (como el recall en casos de refuerzo) y, sobre todo, de un esquema estricto de supervisión humana, garantizando que la IA actúe siempre como una herramienta de apoyo al docente y nunca como un sustituto de su criterio pedagógico.

Referencias

Google. (2026). *Google Colab* [Software]. https://colab.research.google.com/

Anthropic. (2025). *Claude* [Modelo grande de lenguaje]. https://claude.ai

Google. (2026). Gemini [Modelo grande de lenguaje]. https://gemini.google.com/

Kumar, R. (2024, noviembre 7). El Algoritmo A*: Guía completa. Datacamp.com. https://www.datacamp.com/es/tutorial/a-star-algorithm